

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

PROJECT REPORT

ONLINE EFFECTIVE SHOPPING
"SHOP-CORNER"

UNIVERSITY INSTITUTE OF ENGINEERING

TAKSHASHILA

COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY

Souvik Maji (25BCS12160)

Adity Kumar (25BCS10024)

Jatin Gariya (25BCS10453)

Sec / Group : 25BCS – 802 – B

SUBMITTED TO

Mr. Tauseef Sir

CERTIFICATE

This is to certify that the team project entitled "**ONLINE EFFECTIVE SHOPPING SHOP-CORNER**" has been prepared and submitted by **Souvik** (UID: 25BCS12160), **Adity** (UID: 25BCS10024) and **Jatin** (UID: 25BCS10453) as part of the academic requirements for the subject Advance Data Structure (Subject Code: 25CSH202), Semester 3rd, Session July–December 2026.

The project demonstrates the use of AVL trees, Heap, Hash Table, Queue through a product catalogue that supports adding, removing and updating products, searching by ID, name and price, and sorted listing. The work is presented for academic evaluation.

We wish the students success in their future academic and professional endeavours.

Signature of Project Guide

_____

Mr. Tauseef

Technical Trainer

University Institute of Computer Science & Engineering - Takshashila

Chandigarh University

Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Mr. Tauseef, Technical Trainer**, for his valuable guidance, continuous support, and encouragement throughout the development and successful completion of our team project.

We are also grateful to Chandigarh University for providing us with the necessary resources, facilities, and learning environment to undertake and complete this project as part of the subject Advanced Data Structures (Subject Code: 25CSH-202), Semester III, Session July–December 2026.

This project has been a valuable learning experience, allowing us to apply our theoretical knowledge of data structures and algorithms to practical problem-solving.

Finally, we extend our heartfelt thanks to everyone who contributed directly or indirectly through their support, suggestions, and encouragement towards the successful completion of this project.

ABSTRACT

Shop-Corner – E-Commerce Platform with Smart Inventory Management is a full-stack web application developed as part of the subject Advanced Data Structures (25CSH-202). The project aims to simplify online shopping and inventory management by bringing customer shopping experiences and shop-owner operations together on a single platform.

The application provides customers with features such as product browsing, searching, filtering, shopping cart management, checkout, and order history. It also offers shop owners a dedicated dashboard to manage products, monitor stock levels, track orders, view inventory statistics, and manage restocking activities.

The key highlight of this project is the practical implementation of advanced data structures to improve system efficiency. An AVL Tree supports price-based searching, a Hash Table enables faster product lookup, a Trie provides product autocomplete, a Min Heap prioritises low-stock products, and a FIFO Queue manages restocking requests.

Built using React, TypeScript, Fast API, Python, C++17, and Supabase, the application integrates modern web technologies with efficient algorithmic solutions.

Overall, Shop-Corner demonstrates how advanced data structures can solve practical problems in e-commerce, making shopping more convenient and inventory management smarter, faster, and more organised.

INDEX

S.NO	Particulars	Page No.
1	Acknowledgement	2
2	Abstract	3
3	Introduction	5
4	Problem Statement	6
5	Objectives	7
6	Methodology	8-9
7	Technologies Used	10-11
8	Data Structures and Algorithms Used	11-15
9	Algorithm and Flowchart	16-19
10	Screenshots and Output	20-22
11	Conclusion	23
12	References	23

INTRODUCTION

In today's rapidly growing digital world, e-commerce has changed the way people shop and businesses operate. However, managing a large number of products, maintaining accurate stock records, processing orders, and monitoring product availability can be challenging for shop owners. This creates a need for an efficient system that combines online shopping with smart inventory management.

Shop-Corner is a full-stack e-commerce platform designed to simplify shopping and inventory operations through a single application. Customers can browse products, search and filter items, add products to their cart, place orders, and track their purchases. Shop owners can use a dedicated dashboard to manage products, update stock, monitor inventory levels, view business statistics, and handle customer orders.

The major highlight of this project is the practical implementation of Advanced Data Structures and Algorithms (ADSA) to improve operational efficiency. An AVL Tree is used for price-based searching, a Hash Table for faster product lookup, a Trie for product autocomplete, a Min Heap for prioritising low-stock products, and a FIFO Queue for managing restocking requests. These data structures demonstrate how algorithmic concepts can solve real-world problems.

The application is developed using modern technologies such as React, TypeScript, Python, FastAPI, C++17, and Supabase, combining an interactive frontend with backend services and database-driven operations. It also supports three user roles: Admin, Shop Owner, and Customer, each with access to relevant functionalities.

Overall, Shop-Corner aims to bridge the gap between theoretical knowledge and practical software development. It demonstrates how advanced data structures can be integrated with modern web technologies to create a convenient shopping experience and a smarter, faster, and more organised inventory management system.

Project Links

- **GitHub Repository:** [https://github.com/BusySouvik/ADSA-Mini-Project]
- **Live Website:** [https://www.trikstarevitrace.in]

PROBLEM STATEMENT

With the rapid growth of e-commerce, managing an online store has become more complex due to the increasing number of products, customers, and daily transactions. Shop owners often face difficulties in maintaining accurate inventory records, monitoring product availability, processing customer orders, and managing restocking activities. Similarly, customers may experience difficulties in finding specific products, exploring available options, and tracking their purchases efficiently.

Traditional inventory management approaches often involve manual operations or basic searching techniques, which may become inefficient when handling large amounts of product data. This creates a need for a centralised platform that combines online shopping with efficient inventory management using appropriate data structures and algorithms.

The major problems identified in the existing approach are:

- **Inefficient Product Searching:** Searching for a particular product among thousands of available items can take considerable time without suitable searching mechanisms.
- **Inventory Management Difficulties:** Manually maintaining product details, stock quantities, and availability records can lead to inaccurate information and inconsistencies.
- **Low-Stock Detection:** Shop owners may fail to identify products that are running out of stock at the right time, resulting in delayed restocking and missed sales opportunities.
- **Order Processing Challenges:** Managing customer orders, updating purchase records, and maintaining accurate inventory during transactions requires proper coordination.
- **Lack of Smart Search Features:** Basic product searching may not provide autocomplete or prefix-based suggestions, making product discovery less convenient for customers.
- **Unorganised Restocking Operations:** Without a structured process, handling multiple restocking requests can become confusing and difficult to prioritise.
- **Lack of Centralised Management:** Customer shopping activities and shop-owner operations require a common platform where information can be managed securely and efficiently.

OBJECTIVES

The main objective of Shop-Corner is to develop an efficient, user-friendly e-commerce platform integrated with smart inventory management by applying advanced data structures and algorithms to real-world shopping and business operations.

The specific objectives of the project are:

1. Develop an Integrated E-Commerce Platform: Create a single platform where customers can browse products, make purchases, and track their orders, while shop owners can manage their store operations.

2. Implement Efficient Product Searching: Use suitable data structures to make product searching, filtering, sorting, and identification faster and more organised.

3. Apply Advanced Data Structures: Implement AVL Trees, Hash Tables, Tries, Min Heaps, and FIFO Queues to improve the efficiency of different application operations.

4. Improve Inventory Management: Provide shop owners with facilities to add, update, delete, and monitor products along with their available stock quantities.

5. Enable Smart Low-Stock Monitoring: Identify products with low inventory levels and prioritise them for timely restocking using a Min Heap.

6. Enhance Customer Experience: Provide an interactive shopping interface with product autocomplete, search, filters, cart management, and order history.

7. Implement Secure User Authentication: Provide role-based access for Admin, Shop Owner, and Customer to ensure that users can access their respective functionalities.

8. Manage Orders and Restocking: Maintain organised order records and use a FIFO Queue to process restocking requests systematically.

9. Integrate Database Management: Use Supabase to maintain persistent records of users, products, inventory, and orders.

10. Demonstrate Practical ADSA Applications: Apply theoretical concepts of advanced data structures to real-world e-commerce operations and understand their role in developing efficient and scalable applications.

METHODOLOGY

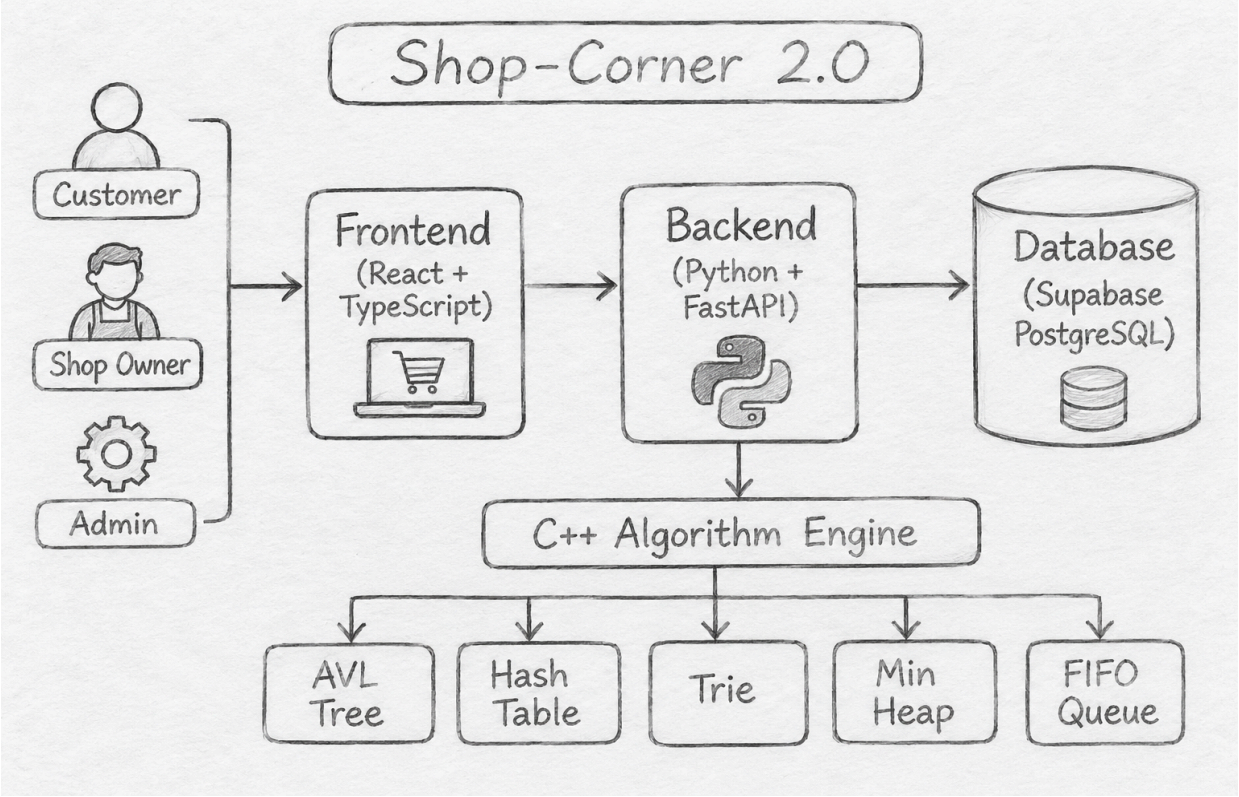
The development of Shop-Corner follows a systematic and structured methodology that integrates web development, database management, and advanced data structures. The project is divided into different phases to ensure proper planning, implementation, integration, and testing of the application.

Requirement Analysis

- Identify the requirements of customers, shop owners, and administrators.
- Analyse the essential features, including product searching, shopping cart, order management, inventory tracking, and restocking.
- Determine the appropriate data structures required for efficient operations.

System Design and Architecture

- Design the overall system architecture by dividing the application into frontend, backend, database, and algorithm engine.
- Define communication between different components.
- Plan separate interfaces and functionalities for different user roles.



Frontend Development

- Develop the user interface using React, TypeScript, and Vite.
- Create customer shopping pages and shop-owner dashboards.
- Implement product browsing, searching, filtering, cart management, and order history.
- Design a responsive and interactive interface for better usability.

Backend Development

- Develop the backend using Python and FastAPI.
- Create APIs for product management, user operations, inventory updates, and order processing.
- Implement business logic and coordinate communication between the frontend, database, and algorithm engine.

Database Design and Integration

- Use Supabase PostgreSQL for storing and managing application data.
- Maintain records of users, products, inventory, and customer orders.
- Integrate Supabase Authentication for secure login, registration, and role-based access.

Implementation of Advanced Data Structures

- AVL trees
- Tries
- Queue
- Heap (Min)
- Hash table

System Integration

- Connect the React frontend with the FastAPI backend through API requests.
- Establish communication between FastAPI and the C++17 algorithm engine using standard input and output.
- Integrate database operations with the application modules.
- Ensure proper synchronisation of product, order, and inventory information.

TECHNOLOGIES USED

The development of Shop-Corner involves a combination of modern programming languages, frameworks, database technologies, and development tools. These technologies are integrated to build a responsive, secure, and efficient e-commerce platform with smart inventory management capabilities.

Technology Stack

The following technologies are used in the development of Shop-Corner:

Frontend Technologies

- React.js: Used to develop interactive and reusable user interfaces.
- TypeScript: Provides type safety and structured frontend development.
- Vite: Used as a fast frontend development server and build tool.
- Tailwind CSS: Used for responsive design and interface styling.

Backend Technologies

- Python: Used for server-side programming and business logic.
- FastAPI: Used to develop REST APIs and handle application requests.
- HTTPX: Supports HTTP communication between application components.

Algorithm Engine

- C++17: Used to implement advanced data structures and algorithms for efficient product searching and inventory operations.

Database Technologies

- Supabase: Provides database services and backend integration.
- PostgreSQL: Stores user details, product information, inventory records, and customer orders.

Authentication

- Supabase Auth: Handles user registration, login, authentication, and session management.

Development Tools

- Visual Studio Code: Used for writing, editing, and managing project code.

- Git & GitHub: Used for version control and source code management.
- npm: Used for managing frontend packages and dependencies.

Deployment

- Render: Used for hosting and deploying the backend application.
- Vercel: Used for frontend hosting

Summary

The selected technology stack combines modern full-stack development with efficient algorithmic techniques. This integration enables Shop-Corner to provide a structured shopping experience, reliable inventory operations, secure user access, and efficient product management while demonstrating the practical application of Advanced Data Structures and Algorithms.

DATA STRUCTURE AND ALGORITHM USED

For your ADSA Mini Project – Shop-Corner, the report should explain not only which data structures are used, but also why they were selected, how they work together, and their time complexities.

Below is a report-ready section following the structure and technical depth of your reference. It focuses on the actual e-commerce and smart inventory management requirements.

Why These Data Structures?

An e-commerce and inventory management system handles multiple operations, such as product searching, sorting, stock updates, order processing and customer management. Using a single data structure for all these operations would not be efficient.

Therefore, different data structures are selected according to the requirements of the system.

Data Structure	Search	Insertion	Deletion	Main Application
Array / Vector	O(n)	O(1) amortized at end	O(n)	Temporary product lists
Hash Table	O(1) average	O(1) average	O(1) average	Fast product lookup
Linked List	O(n)	O(1) at head	O(1) at head	Maintaining sequential records
Queue	O(n)	O(1)	O(1)	Order processing
Stack	O(n)	O(1)	O(1)	Undo and recent operations
AVL Tree	O(log n)	O(log n)	O(log n)	Product indexing and sorted searches
Graph	O(V + E) traversal	O(1) adjacency insertion (typical)	Depends on representation	Product/category relationships

AVL Tree – Product Indexing

The AVL Tree is the primary data structure used for maintaining product records in the system. Each node stores a product key, a reference to the product record, pointers to its left and right children, and its height.

The tree maintains the Binary Search Tree property:

- Left subtree contains smaller keys.
- Right subtree contains larger keys.
- The height difference between the left and right subtrees is at most 1.

Multiple Indexing Using AVL Trees

To support different product search requirements, AVL Trees can maintain separate indexes over the same product records.

Each index serves a different purpose:

1. ID Tree: Quickly identifies products using their unique product IDs.
2. Name Tree: Supports alphabetical product listing and name-based searching.
3. Price Tree: Helps retrieve products within a specified price range.

Using multiple indexes avoids repeatedly sorting the entire product collection for different queries.

Hash Table – Fast Product Lookup

A Hash Table stores product information using a unique key and a hash function.

It provides fast access to product records without requiring traversal of an entire collection.

Working:

1. A product ID is supplied as the search key.
2. The hash function calculates the corresponding index.
3. The system accesses the stored product record.
4. If a collision occurs, a collision-resolution technique is used.

Application in Shop-Corner:

- Quick product identification.
- Checking whether a product already exists.
- Accessing product information using its ID.
- Reducing repeated sequential searches.

4.5 Queue – Order Processing

A Queue follows the FIFO (First In, First Out) principle, meaning the first order received is processed first.

It is useful for managing pending customer orders.

Working:

1. A customer places an order.
2. The order is added to the rear of the queue.
3. The system processes the order from the front.
4. Once processed, the order is removed from the queue.

Application in Shop-Corner:

- Maintaining pending orders.
- Processing orders in arrival order.
- Managing order-processing workflows.
-

Trie – Prefix-Based Product Searching

A Trie (Prefix Tree) is a tree-based data structure used to store and search strings efficiently. It is particularly useful for implementing product search suggestions and autocomplete functionality.

Working:

1. Each node represents a character.
2. Words are stored character by character.
3. Words sharing the same prefix share common nodes.
4. When a user types a prefix, the Trie retrieves matching product names.

Application in Shop-Corner:

- Product name autocomplete.
- Search suggestions while typing.

Heap – Priority-Based Inventory and Order Management

A Heap is a complete binary tree that follows the heap property. It is useful when the system needs to identify the highest- or lowest-priority element efficiently.

Min Heap - Smallest element remains at the root.

Working:

1. A new element is inserted into the heap.
2. The heap property is restored using heapify-up.
3. The highest- or lowest-priority element is available at the root.
4. After removal, heapify-down restores the heap property.

Application in Shop-Corner 2.0:

- Identifying products with critically low stock.
- Prioritizing inventory restocking.
- Processing high-priority orders.

Data Structure	**Role in Shop-Corner**
AVL Tree	Balanced product indexing
Hash Table	Fast product lookup
Trie	Product autocomplete and prefix search
Min Heap	Low-stock product prioritization
Queue	Pending order processing

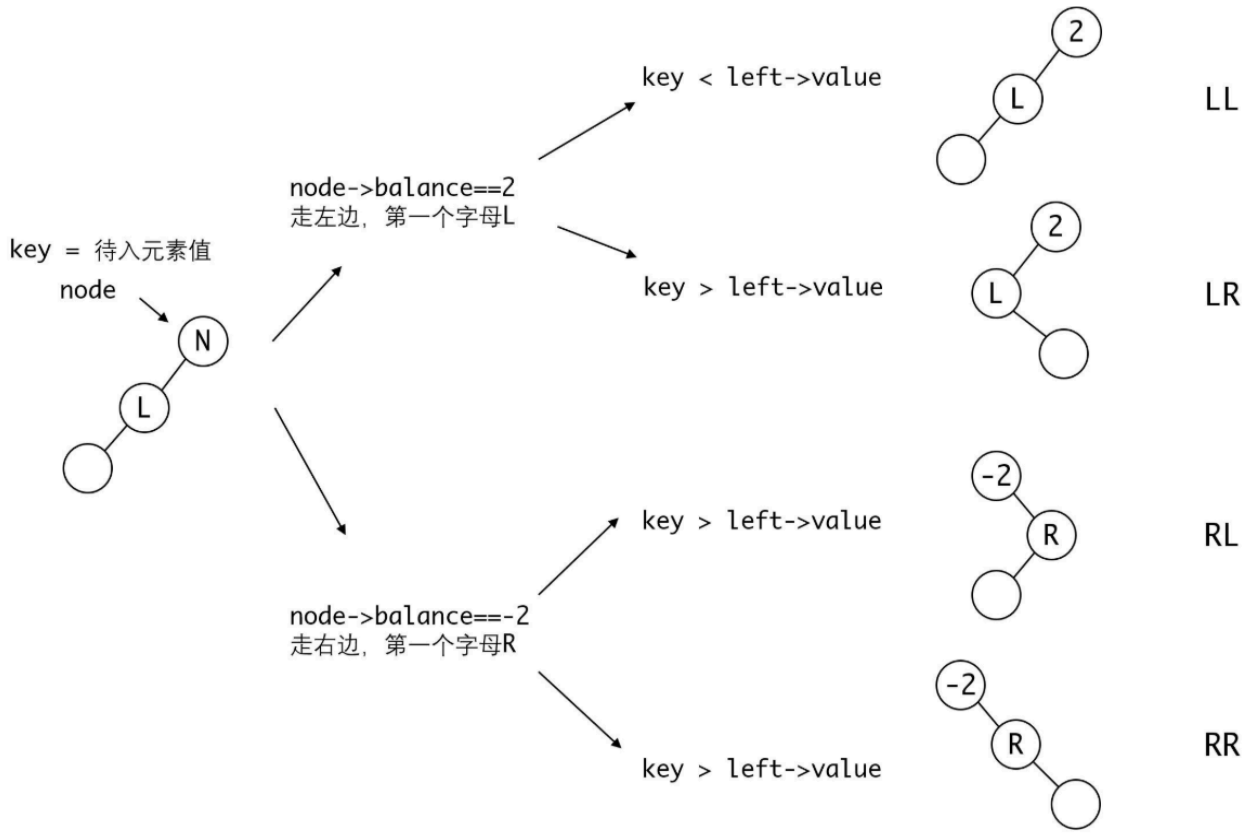
ALGORITHM AND FLOWCHART

Algorithm 1: AVL Tree – Product Insertion

Objective: To insert a new product into the AVL Tree while maintaining its balanced structure.

Algorithm:

1. Start.
2. Create a new product node.
3. Compare the product key with the current node.
4. If the key is smaller, move to the left subtree.
5. If the key is greater, move to the right subtree.
6. Insert the product at the appropriate empty position.
7. Update the height of each affected node.
8. Calculate the balance factor.
9. Identify the imbalance condition (LL, RR, LR or RL).
10. Perform the required rotation.
11. Return the balanced AVL Tree.
12. Stop.

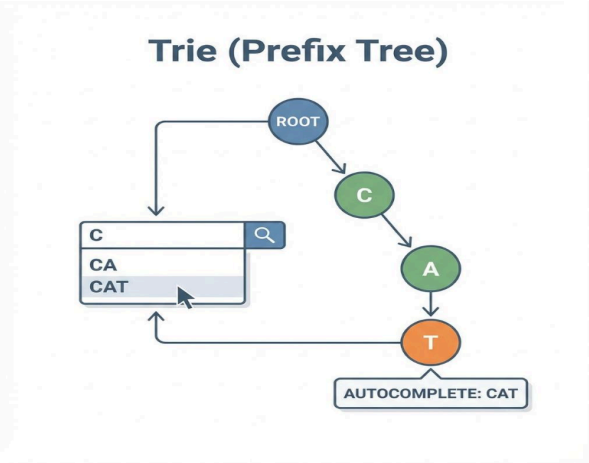


Algorithm 2: Trie – Product Autocomplete

Objective: To suggest products based on the prefix entered by the customer.

Algorithm:

1. Start.
2. Accept the search prefix from the customer.
3. Convert the prefix into lowercase.
4. Start from the root node of the Trie.
5. Traverse the Trie character by character.
6. If a character is not found, display no suggestions.
7. If the prefix exists, explore its descendant nodes.
8. Collect all complete product names.
9. Display matching product suggestions.
10. Stop.



Algorithm 3: Min Heap – Low Stock Prioritization

Objective: To identify and prioritize products that require restocking based on their available stock.

Algorithm:

1. Start.
2. Retrieve product stock information.
3. Check each product against the minimum stock threshold.
4. If stock is below the threshold, insert the product into the Min Heap.
5. Assign priority according to available stock.
6. Apply heapify-up after insertion.

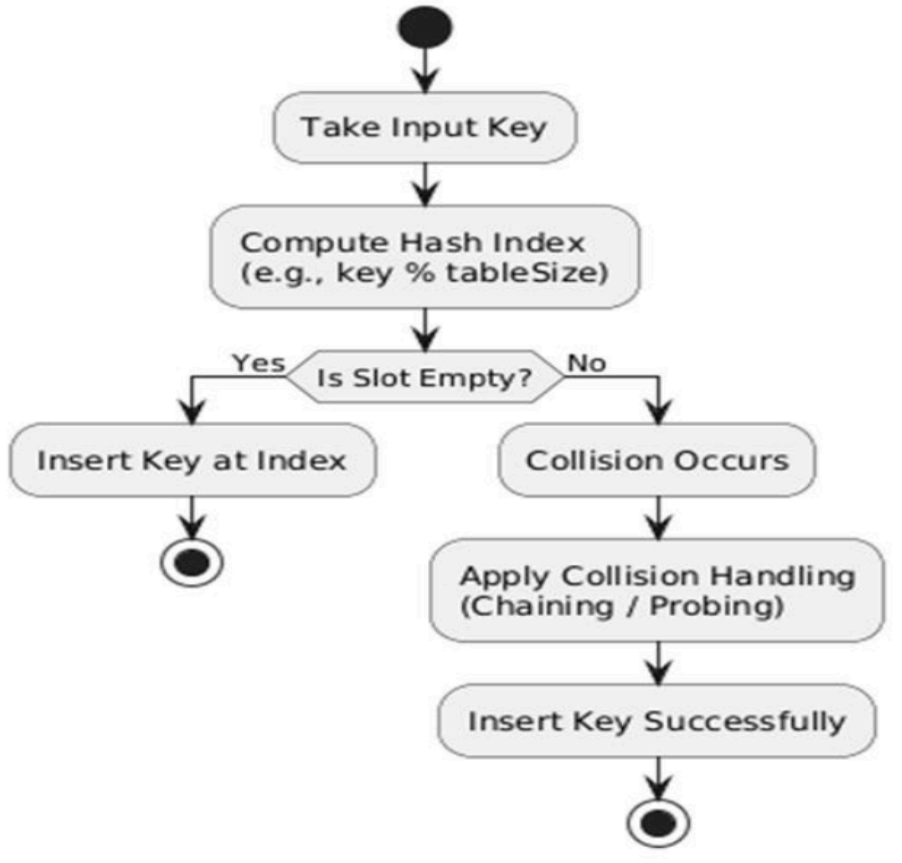
7. Retrieve the root element containing the lowest-stock product.
8. Display products in restocking priority order.
9. Continue until the heap becomes empty.
10. Stop.

Algorithm 4: Hash Table – Product Search

Objective: To retrieve product details quickly using a unique product ID.

Algorithm:

1. Start.
2. Accept the product ID.
3. Apply the hash function to calculate the index.
4. Access the corresponding hash table location.
5. Check whether the product exists.
6. If found, retrieve and display product details.
7. Otherwise, handle collisions or display product not found.
8. Stop.

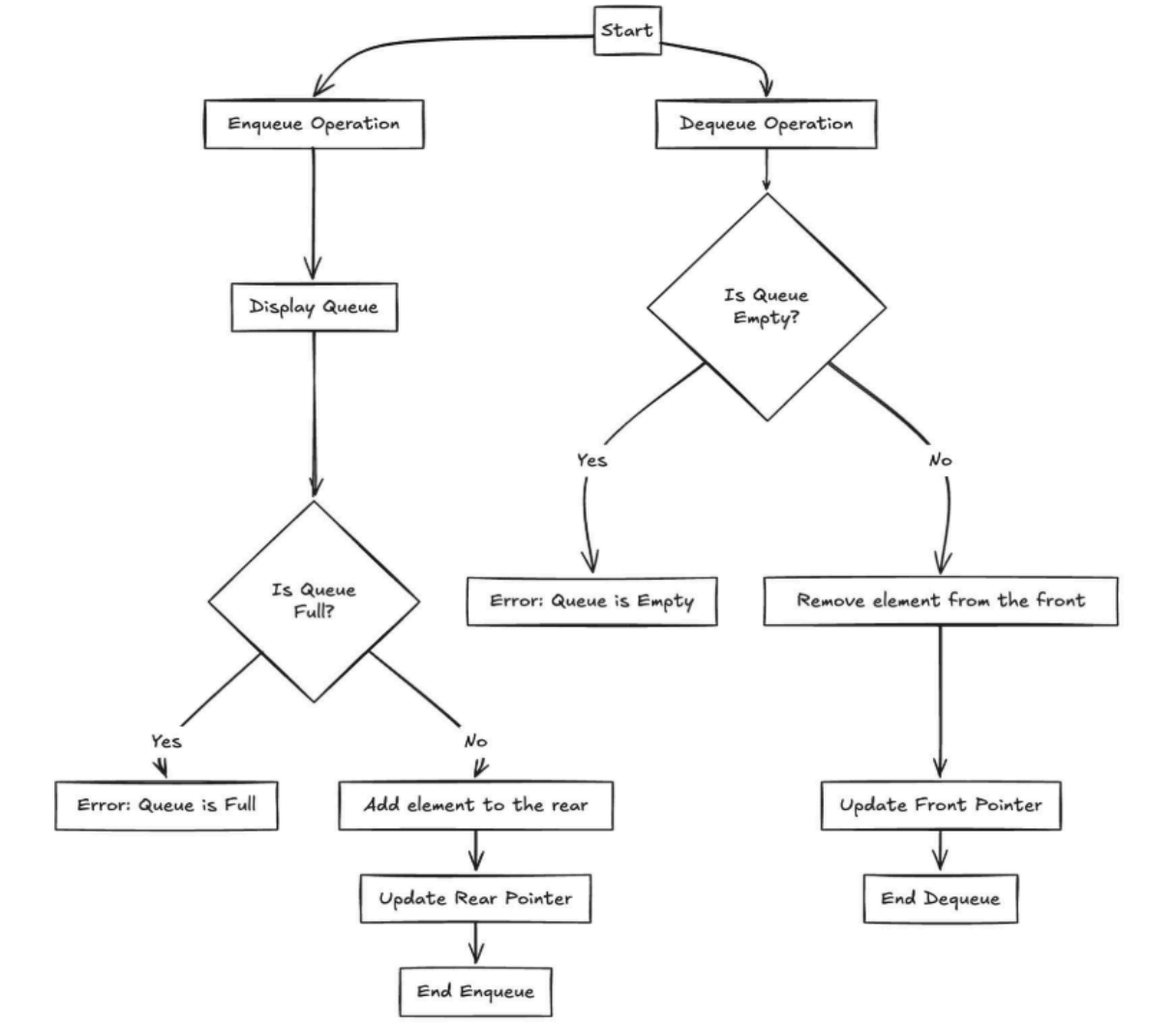


Algorithm 5: Queue – Order Processing

Objective: To process customer orders according to the First In First Out (FIFO) principle.

Algorithm:

1. Start.
2. Receive a new customer order.
3. Insert the order at the rear of the queue.
4. Check whether the queue is empty.
5. If the queue is not empty, retrieve the front order.
6. Process the order.
7. Remove the processed order from the queue.
8. Repeat until no pending orders remain.
9. Stop.



SCREENSHOTS AND OUTPUTS

ADSA-Mini-Project / cpp_engine / src / main.cpp

BusySouvik Shop-Corner ADSA mini-project

Code Blame 78 lines (64 loc) · 2.33 KB

```cpp
#include "engine.hpp"
#include <iomanip>

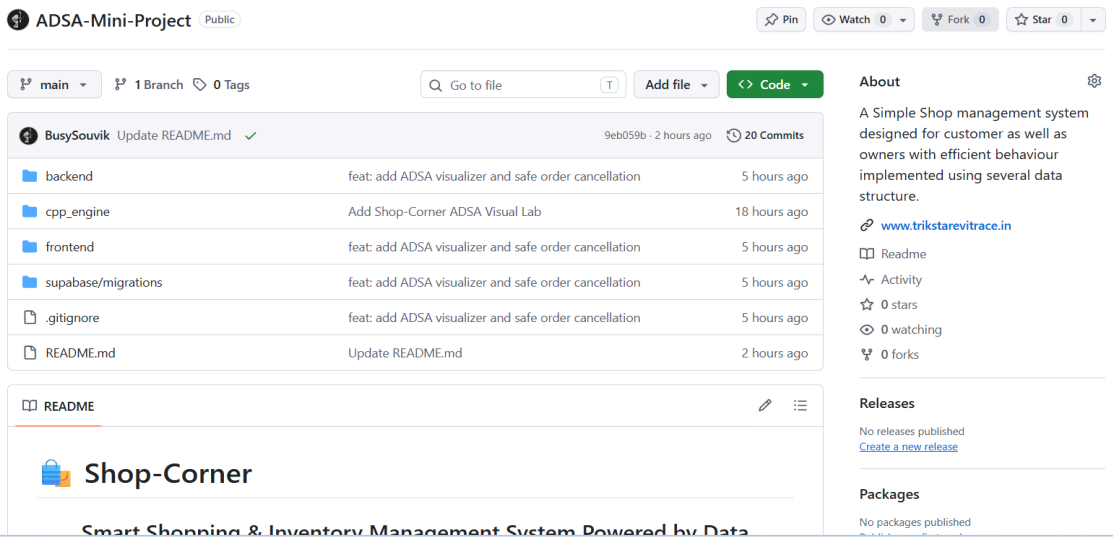
int main() {
    std::vector<Product> products = {
        {"demo-001", "P101", "Laptop", 55000, 3},
        {"demo-002", "P102", "Wireless Mouse", 799, 20},
        {"demo-003", "P103", "Mechanical Keyboard", 2499, 2},
        {"demo-004", "P104", "Laptop Stand", 1299, 5},
        {"demo-005", "P105", "USB Hub", 999, 1}
    };

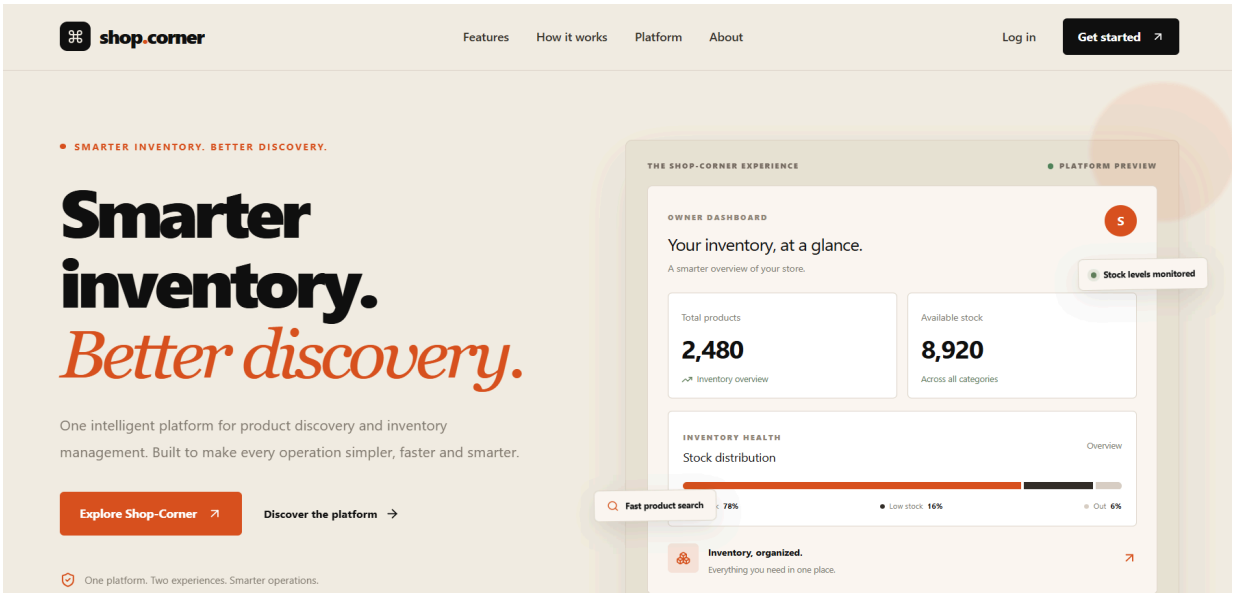
    std::cout << "\n==================================\n";
    std::cout << "       SHOP-CORNER ADSA ENGINE\n";
    std::cout << "==================================\n";

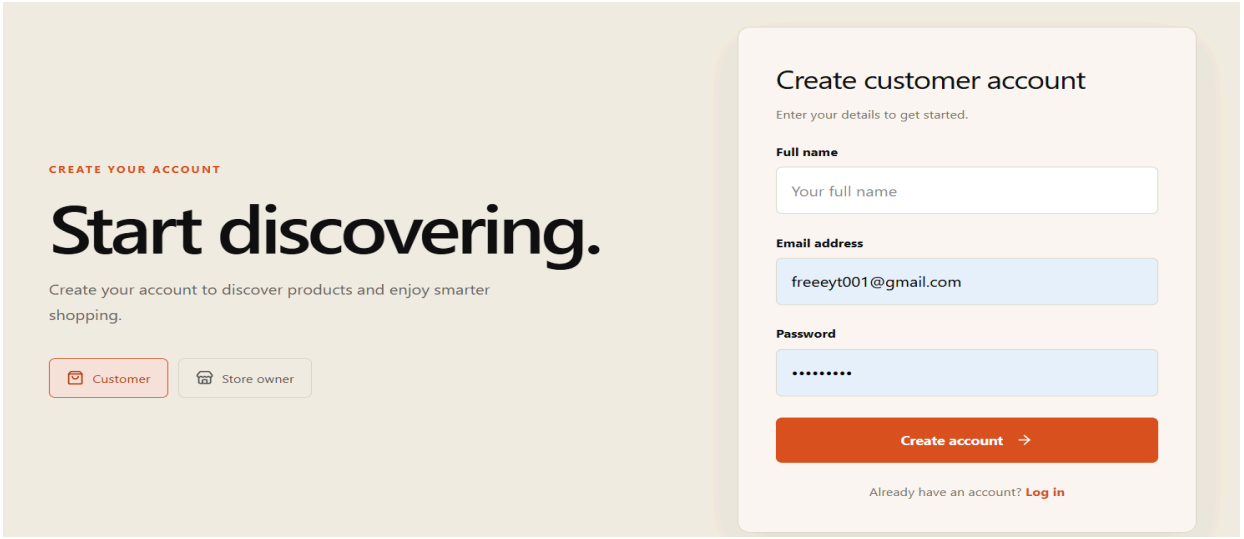
    // AVL Tree
    AVLTree avl;
    for (const auto& p : products)
        avl.insert(p.price, p.id);

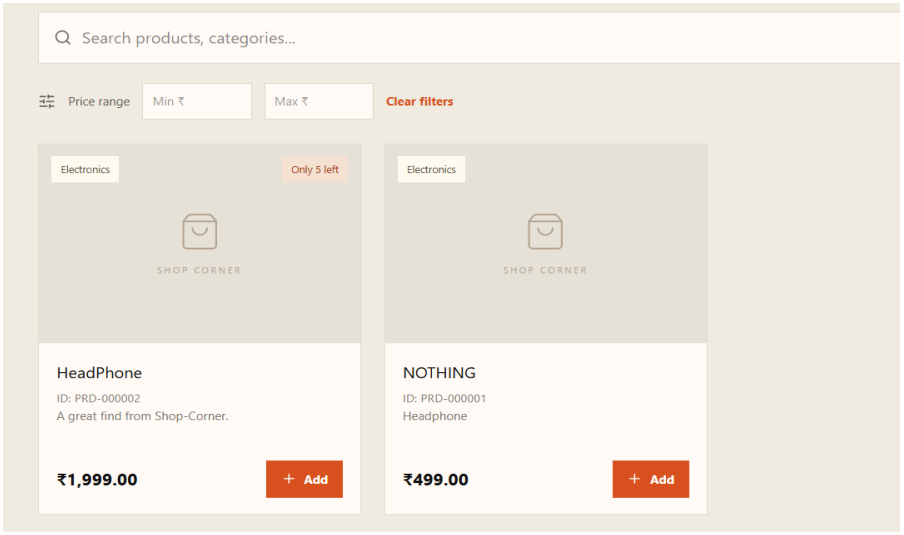
    std::cout << "\n1. AVL TREE - Products priced 500 to 3000:\n";
    auto priceResults = avl.searchPriceRange(500, 3000);
    for (const auto& id : priceResults)
        std::cout << "   " << id << '\n';

    // Hash Table
    ProductHashTable hashTable;
```

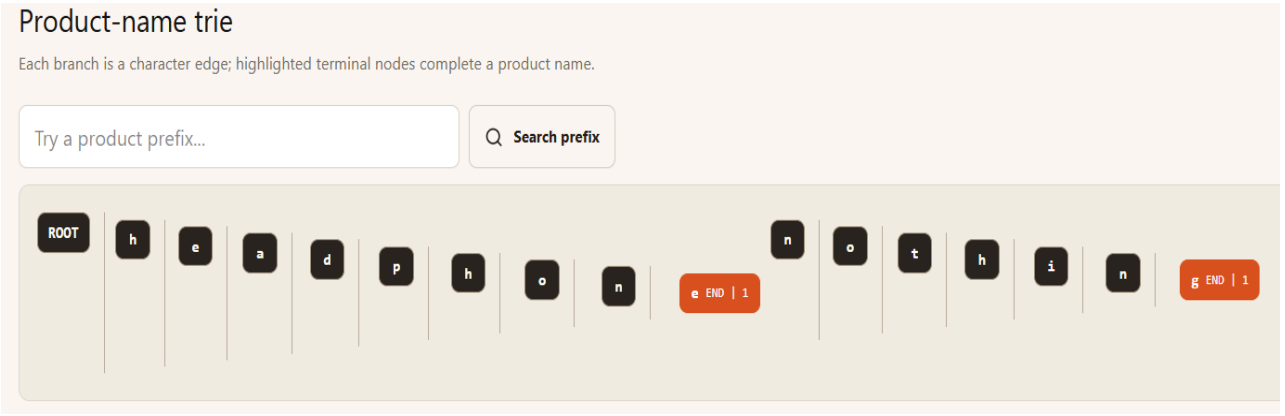


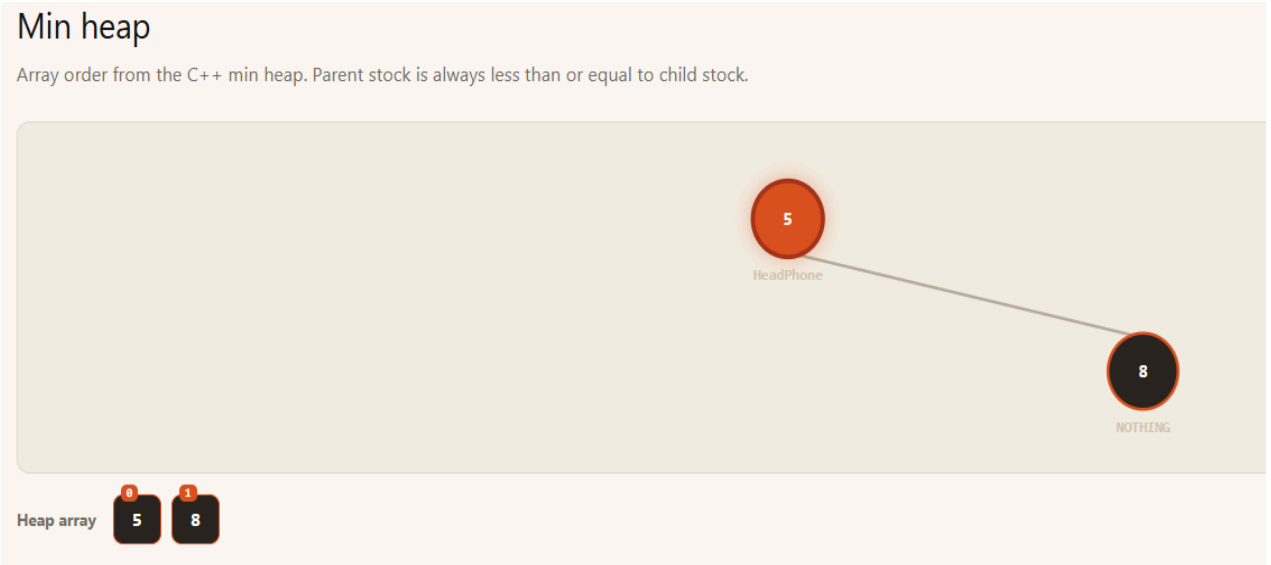

shop.corner
Features
How it works
Platform
About
Log in
Get started
SMARTER INVENTORY. BETTER DISCOVERY.
Smarter inventory.
Better discovery.
One intelligent platform for product discovery and inventory management. Built to make every operation simpler, faster and smarter.
Explore Shop-Corner
Discover the platform
One platform. Two experiences. Smarter operations.
THE SHOP-CORNER EXPERIENCE
PLATFORM PREVIEW
OWNER DASHBOARD
Your inventory, at a glance.
A smarter overview of your store.
Stock levels monitored
Total products
2,480
Inventory overview
Available stock
8,920
Across all categories
INVENTORY HEALTH
Stock distribution
Overview
Fast product search
78%
Low stock 16%
Out 6%
Inventory, organized.
Everything you need in one place.


CREATE YOUR ACCOUNT
Start discovering.
Create your account to discover products and enjoy smarter shopping.
Customer
Store owner
Create customer account
Enter your details to get started.
Full name
Your full name
Email address
freeeyt001@gmail.com
Password
Create account
Already have an account? Log in


Search products, categories...
Price range
Min ₹
Max ₹
Clear filters
Electronics
Only 5 left
SHOP CORNER
HeadPhone
ID: PRD-000002
A great find from Shop-Corner.
₹1,999.00
Add
Electronics
SHOP CORNER
NOTHING
ID: PRD-000001
Headphone
₹499.00
Add


ADSA Visual Lab
Live structure snapshots from your store's C++ inventory engine.
Refresh structures
Products indexed
2
AVL rotations
0
Hash capacity
101
Pending FIFO requests
1
AVL Tree
Hash Table
Trie
Min Heap
FIFO Queue
01 / SELF-BALANCING SEARCH TREE
AVL price index
Nodes show price, height and balance factor. Edges are actual parent-child links.
Previous
Next rotation
499
h:2 | bf:-1
1,999
h:1 | bf:0
Insertion rotation log
No rotations were needed while indexing this inventory.


Product-name trie
Each branch is a character edge; highlighted terminal nodes complete a product name.
Try a product prefix...
Search prefix
ROOT
h
e
a
d
p
h
o
n
e END | 1
n
o
t
h
i
n
g END | 1


Min heap
Array order from the C++ min heap. Parent stock is always less than or equal to child stock.
5
HeadPhone
8
NOTHING
Heap array
0
5
1
8

CONCLUSION

The Shop-Corner project successfully demonstrates the practical application of Advanced Data Structures and Algorithms in developing an e-commerce and smart inventory management system. It combines shopping functionalities with efficient product searching, inventory management, and order processing.

The use of data structures such as AVL Trees, Tries, Heaps, Hash Tables, Queues, and Stacks helps improve the efficiency of different operations and provides practical understanding of algorithm performance and time complexity.

Overall, the project bridges the gap between theoretical concepts and real-world software development. It also provides scope for future improvements, such as better product recommendations, inventory prediction, and enhanced order management.

REFERENCES

1. React Documentation: React – The library for web and native user interfaces.

2. TypeScript Documentation: TypeScript Handbook.

3. Vite Documentation: Vite – Next Generation Frontend Tooling.

4. FastAPI Documentation: FastAPI Official Documentation.

5. Supabase Documentation: Supabase Database, Authentication and Backend Service.

6. PostgreSQL Documentation: PostgreSQL Official Documentation.

7. Python Documentation: Python Official Documentation.

10. C++ Reference: C++ Language and Standard Library Reference.

Source Code: Shop-Corner – https://github.com/BusySouvik/ADSA-Mini-Project

Live Application: Shop-Corner – https://www.trikstarevitrace.in